CodeFlow: Automating the Flow of Code with LLMs

Marino, Erik Bran
erik.marino@uevora.pt
Universidade de Évora, Portugal

Bassi, Davide
davide.bassi@usc.es
Universidade de Santiago de Compostela, Spain

Baleato, Suso
jesusmanuel.benitez.baleato@usc.gal
Universidade de Santiago de Compostela, Spain

Vieira, Renata
renatav@uevora.pt
Universidade de Évora, Portugal

Abstract

Researchers increasingly rely on computational methods for analyzing large-scale textual data. However, implementing these methods often requires programming skills that may lie outside their core expertise. We present CodeFlow, a system that automates code generation and refinement through iterative dialogue with large language models (LLMs). Our architecture translates high-level research goals into functional code through automated debugging cycles. We demonstrate its practical utility through a case study on sentiment analysis, where it successfully automated the implementation of a BERT-based classifier achieving 0.95 accuracy. By handling technical implementation details automatically, CodeFlow enables researchers to focus on their research questions while maintaining high standards of computational rigor. While currently implemented with GPT-4o, the system's architecture is designed to be adaptable to any LLM capable of processing both natural language and code.

Keywords: NLP, LLMs, Digital Humanities, Code Generation, Open Science

Introduction

Digital humanities and computational social sciences are rapidly evolving, employing computational methods to analyze complex cultural and textual phenomena (Lazer et al. 2009). Natural language processing, particularly through large language models, has become instrumental in these fields (Jurafsky / Martin 2019), despite documented limitations (Bender et al. 2021). The implementation of these methods often requires programming expertise, creating a significant barrier for researchers whose primary focus is qualitative and cultural analysis. While recent advances in code generation tools like GitHub Copilot (Chen et al. 2021) and LLMs like GPT-4 (Moussiades / Zografos 2023) offer promising solutions, their outputs frequently require substantial manual refinement (Poldrack et al. 2023). Researchers often find themselves in a tedious cycle of copying error logs back to these models and implementing suggested fixes—a process ripe for automation and particularly challenging for those without extensive programming backgrounds. Several specialized tools exist for specific analytical tasks, such as VADER (Hutto / Gilbert 2014) for sentiment analysis or general text processing libraries. However, these tools often require customization or adaptation to specific research questions, creating a gap between available tools and research needs. We present CodeFlow, a system that addresses these challenges through:

- An automated pipeline that handles code generation and refinement with LLMs
- A purpose-driven architecture that translates research goals into functional code
- An iterative refinement process that automatically handles common programming issues
- A simple interface allowing researchers to define goals in natural language

Methods

CodeFlow's architecture comprises four integrated components designed to work together in a pipeline (Figure 1):

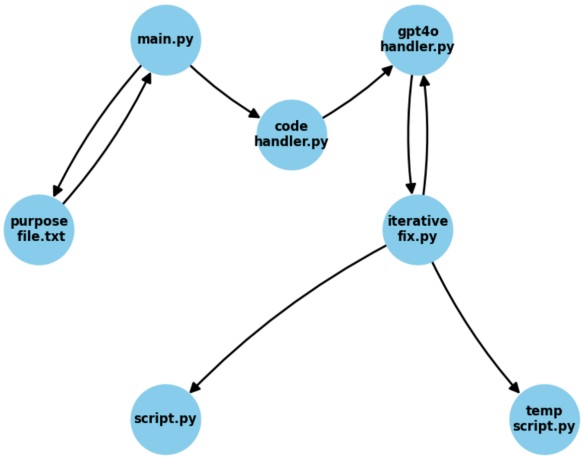

Figure 1. Pipeline diagram of the code automation system.

- **GPT-4o Handler**: Manages communication with OpenAI's API, maintaining context across interactions to ensure coherent code generation and refinement. This component structures the dialogue with the LLM to optimize code quality and consistency. While currently optimized for GPT-4o, this component could be adapted to work with any LLM that provides an API for code generation.
- **Iterative Fix Module**: Implements the core refinement loop by:
  - Executing generated code and capturing outputs
  - Analyzing execution results and error patterns
  - Automatically requesting and implementing refinements
  - Generating visual outputs for user review
- **Code Handler**: Manages code execution and modification by:
  - Parsing and validating generated code
  - Identifying optimization opportunities
  - Handling common error patterns
- **Main Orchestrator**: Coordinates the overall workflow by:
  - Processing the user's purpose.txt file
  - Managing component interactions
  - Tracking progress toward specified goals

The system follows a structured workflow:

1. User defines goals in purpose.txt
2. System generates initial code implementation
3. Iterative Fix Module executes and evaluates code
4. Results and logs are automatically fed back for refinement
5. Process continues until goals are met or human intervention is needed

Use Cases and Evaluation

We evaluated CodeFlow's effectiveness on a standard sentiment analysis benchmark task. Using datasets from Amazon, IMDB, and Yelp (Yelp 2020, Maas et al. 2011, Kotzias et al. 2015), the system automated the entire implementation process from data preprocessing to model fine-tuning and evaluation. The automated implementation achieved robust performance metrics:

- Accuracy: 0.95
- Precision: 0.92
- Recall: 0.98
- F1 Score: 0.95

The system handled key implementation challenges including:

- Data preprocessing and cleaning
- Model architecture configuration
- Training parameter optimization
- Performance evaluation with appropriate metrics

While our current evaluation focused on this benchmark task, we envision potential applications in digital humanities research, such as analyzing sentiment patterns in literary texts, exploring thematic developments across historical documents, or characterizing linguistic features in cultural artifacts. These applications represent promising directions for future work rather than completed studies.

Impact and Comparison with Existing Tools

CodeFlow contributes to making computational methods more accessible to researchers with limited programming expertise (Verduzco et al. 2023). The system can be positioned within the existing ecosystem of tools for computational research.

Positioning within the Computational Research Toolkit

- Comparison with Direct Code Generation Tools: Unlike GitHub Copilot (Chen et al. 2021) or similar code completion tools, CodeFlow manages the entire process of generating, executing, and refining code based on research objectives expressed in natural language, ra-

ther than requiring line-by-line programming guidance.
- Comparison with Pre-built NLP Libraries: While tools like NLTK, spaCy, or HuggingFace provide powerful pre-built components, they still require programming knowledge to implement. CodeFlow bridges this gap by generating the implementation code for these libraries based on research requirements.
- Potential for Digital Humanities: Tools like Voyant Tools (Sinclair / Rockwell 2016) offer accessible interfaces for text analysis but may limit customization. CodeFlow could provide similar accessibility while maintaining the flexibility of custom code implementations.

Key Benefits for Research

The system enables researchers to:

- Implement advanced NLP methods with reduced coding expertise
- Focus on research questions rather than technical implementation
- Maintain reproducible computational workflows
- Access sophisticated analysis techniques previously requiring specialized programming knowledge

By potentially functioning with different LLMs, CodeFlow could offer flexibility in deployment contexts, addressing concerns about cost and accessibility in research environments with limited resources.

Limitations and Future Work

While CodeFlow reduces technical barriers, it does not eliminate the need for all technical expertise. Our evaluation revealed several important limitations:

- Creative Problem-Solving: The system may not match human creativity in developing novel solutions, particularly for unconventional research questions (Verduzco et al. 2023). When facing complex or unique research problems, human intervention and creative thinking remain essential.
- LLM Reliability: Despite our iterative refinement approach, the underlying LLM can still produce incorrect or irrelevant code (Poldrack et al. 2023). Users should remain vigilant about validating outputs and results, particularly for research-critical applications.
- Security Concerns: A critical limitation involves the potential for LLMs to generate code with security vulnerabilities that inexperienced users may not detect (Wang et al. 2024, Islam et al. 2025, Basic / Giaretta 2024). This risk is particularly relevant when the automated code handles sensitive data or when researchers lack the technical expertise to evaluate code security, potentially introducing vulnerabilities into research workflows.
- Technical Knowledge Requirements: While CodeFlow significantly lowers technical barriers, users still benefit from basic programming literacy to:
  - Recognize and address semantic errors in generated code
  - Formulate research goals in terms that can be effectively translated to code
  - Interpret and critically evaluate results
- Technical Boundaries: Some tasks require human intervention, such as:
  - Installing new dependencies
  - Addressing domain-specific requirements
  - Handling complex error patterns

Conclusion and Future Work

CodeFlow demonstrates the potential for automated systems to make computational methods more accessible to digital humanities scholars and other researchers. Future development will focus on:

- Expanding support for humanities-specific research methodologies, including corpus analysis, stylometry, and network analysis of historical and cultural data
- Enhancing error detection and correction capabilities
- Developing more sophisticated automation patterns
- Improving integration with common digital humanities research workflows and existing tools
- Supporting additional LLM backends, with particular emphasis on open-source and locally-deployable models to address privacy concerns, reduce dependency on commercial APIs, and minimize computational costs for research institutions with limited budgets

These directions for future work reflect our commitment to developing tools that effectively bridge technical implementation and humanities research questions. The current implementation of CodeFlow is available as an open-source project on GitHub[1].

Sample Purpose File

Below is an excerpt from a sample purpose.txt file used to initialize our system for a sentiment analysis task:

Task: Sentiment Analysis Dataset:

- amazon_cells_labelled.txt

- imdb_labelled.txt

- yelp_labelled.txt Requirements:

1. Fine-tune BERT model for binary classification

2. Target accuracy: > 90%

3. Handle multi-language input

4. Generate performance metrics

5. Implement cross-validation Additional Parameters:

- Batch size: 32

- Learning rate: 2e-5

- Max sequence length: 128

- Training epochs: 4

This example demonstrates how the system translates high-level research requirements into executable code. For future applications in digital humanities, similar purpose files could specify requirements for analyzing literary texts, historical documents, or cultural artifacts.

Notes

1. https://github.com/erikbranmarino/CodeFlow.

Bibliography

Baayen, Harald / Halteren, Hans van / Tweedie, Fiona (1996): "Outside the cave of shadows: using syntactic annotation to enhance authorship attribution", in: Literary and Linguistic Computing 11, 3: 121–132.

Basic, E. / Giaretta, A. (2024): Large Language Models and Code Security: A Systematic Literature Review. arXiv preprint arXiv:2412.15004.

Bender, E. M. / Gebru, T. / McMillan-Major, A. / Shmitchell, S. (2021): "On the dangers of stochastic parrots: Can language models be too big?", in: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, 610–623. DOI: 10.1145/3442188.3445922.

Biber, Douglas (1988): Variation Across Speech and Writing. Cambridge: Cambridge University Press.

Chen, M. / Tworek, J. / Jun, H. / Yuan, Q. / Oliveira Pinto, H. P. de / Kaplan, J. / Edwards, H. / Burda, Y. / Joseph, N. / Brockman, G. et al. (2021): Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

Ellis, David (1987): The Derivation of a Behavioural Model for Information Retrieval Design. Ph.D. thesis, University of Sheffield.

Garfinkel, Michele S. / Weiss, Steven. C. (1999): "In the court of history, Ehlers v. Bush", in: Recent Science Newsletter 1, 1: 6-7 <http://web.archive.org/web/20030802233236/http://recentscience.gwu.edu/RSN/PDF.files/RSN1.1.pdf> [15.03.2021].

Hazadiah, Mohd Dahan (1993): "Topic as a Dynamic Element in Spoken Discourse", in: Baker, Mona / Francis, Gill / Tognini-Bonelli, Elena (eds.): Text and Technology. In honour of John Sinclair. Amsterdam: John Benjamins 55-74.

Hutto, C.J. / Gilbert, E.E. (2014): "VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text", in: Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media.

Islam, M. R. / Nepal, S. / Alazab, D. / Tong, T. H. / Alazab, M. (2025): "Measurement of the security quality of LLM-generated code: What makes it vulnerable by default?", in: Information and Software Technology, 167.

Jurafsky, D. / Martin, J. H. (2019): Speech and Language Processing. Pearson, 3rd edition.

King, G. (1995): "Replication, Replication", in: PS: Political Science and Politics, 28: 444-452.

Kotzias, D. / Denil, M. / De Freitas, N. / Smyth, P. (2015): "From group to individual labels using deep features", in: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 597–606.

Lazer, D. / Pentland, A. / Adamic, L. / Aral, S. / Barabasi, A.-L. / Brewer, D. / Christakis, N. / Contractor, N. / Fowler, J. / Gutmann, M. et al. (2009): "Computational social science", in: Science, 323(5915): 721–723. DOI: 10.1126/science.1167742.

Maas, A. L. / Daly, R. E. / Pham, P. T. / Huang, D. / Ng, A. Y. / Potts, C. (2011): "Learning word vectors for sentiment analysis", in: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, 1: 142–150.

Moussiades, L. / Zografos, G. (2023): "Openai's gpt4 as coding assistant", in: ArXiv, abs/2309.12732. DOI: 10.48550/arXiv.2309.12732.

Poldrack, R. / Lu, T. / Beguš, G. (2023): "Ai-assisted coding: Experiments with gpt-4", in: ArXiv, abs/2304.13187. DOI: 10.48550/arXiv.2304.13187.

Richardson, Stephan D. / Braden-Harder, Lisa C. (1988): "The Experience of Developing a Large-Scale Natural Language Text Processing System: CRITIQUE", in: Association for Computational Linguistics (ed.): Second Conference on Applied Natural Language Processing, Austin, TX, February 1988: 195–202. DOI: 10.3115/974235.974271.

Sinclair, S. / Rockwell, G. (2016): Voyant tools. URL: https://voyant-tools.org/ [15.03.2021].

Verduzco, J. C. / Holbrook, E. / Strachan, A. (2023): "Gpt-4 as an interface between researchers and computational software: improving usability and reproducibility", in: ArXiv, abs/2310.11458. DOI: 10.48550/arXiv.2310.11458.

Wang, E. / Cao, A. / Sugino, K. / Zeng, K. / Carvalho, G. / Olukotun, K. (2024): LLM4Vuln: A Rigorous Benchmark for Assessing LLMs in Security-Critical Code Generation. arXiv preprint arXiv:2412.15004v1.

Yelp (2020): Yelp dataset challenge. URL https://www.yelp.com/dataset [15.03.2021].